

Phase 3 P0 — Verification Evidence

Project: Vyapar AI Employee | Workspace: Z:\Vyapar AI\cursor | Date: June 1, 2026

9. Code existence confirmation

Answer: B) actually exists in repository

File	Status	Lines
session_state_manager.py	EXISTS	169
turn_router.py	EXISTS	207
response_validator.py	EXISTS	134
language_lock.py	EXISTS	56
memory_validator.py	EXISTS	87
memory_db.py	EXISTS	126
ai_employee_engine.py	EXISTS	467
prompts.py	EXISTS	85
main.py	EXISTS	246
tests/test_p0_phase3.py	EXISTS	138
tests/test_sales_mode.py	EXISTS	52
docs/migrations/001_conversation_state.sql	EXISTS	23

2. Git commit hashes

```
NO_GIT_REPO

Command: cd "Z:\Vyapar AI\cursor"; if (Test-Path .git) { git log --oneline -20 } else { Write-Output "NO_GIT_REPO" }
Output: NO_GIT_REPO
No commit hashes exist.
```

3–6. Test execution

```
Command:
C:\Users\Ncell\AppData\Local\Microsoft\WindowsApps\python3.exe -m unittest discover -s "Z:\Vyapar AI\cursor\tests" -v

Output:
python3.exe : Python was not found; run without arguments to install from the Microsoft Store...
Exit code: 9009

Tests run:    0
Tests passed: 0
Tests failed: 0

Static count of def test_ methods defined in tests/:
test_p0_phase3.py      18
test_sales_mode.py     5
test_sales_objection.py 5
test_lead_phase2.py    7
TOTAL DEFINED          35
```

8. SQLAlchemy ConversationState model (memory_db.py lines 71–90)

```
class ConversationState(Base):
    __tablename__ = "conversation_state"

    user_id: Mapped[str] = mapped_column(String(64), primary_key=True)
    company_id: Mapped[str] = mapped_column(String(64), primary_key=True)
    language: Mapped[str | None] = mapped_column(String(16), nullable=True, default="english")
    name: Mapped[str | None] = mapped_column(String(120), nullable=True)
    phone: Mapped[str | None] = mapped_column(String(32), nullable=True)
    location: Mapped[str | None] = mapped_column(String(160), nullable=True)
    package_interest: Mapped[str | None] = mapped_column(String(160), nullable=True)
    lead_stage: Mapped[str | None] = mapped_column(String(32), nullable=True)
    pitch_count: Mapped[int] = mapped_column(Integer, default=0)
    phone_collected: Mapped[bool] = mapped_column(Boolean, default=False)
    escalation_requested: Mapped[bool] = mapped_column(Boolean, default=False)
    last_assistant_reply: Mapped[str | None] = mapped_column(Text, nullable=True)
    turn_count: Mapped[int] = mapped_column(Integer, default=0)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=False), server_default=func.now())
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=False), server_default=func.now(), onupdate=func.now()
    )
```

7a. phone_collected logic

```
// session_state_manager.py sync_session_state (lines 131-136)
phone = facts.get("phone") or memory.get("phone")
if not phone and lead:
    phone = getattr(lead, "phone", None) or getattr(lead, "contact_value", None)
if phone:
    state.phone = phone
    state.phone_collected = True

// session_state_manager.py mark_phone_collected (lines 159-163)
def mark_phone_collected(user_id: str, company_id: str, phone: str) -> SessionState:
    state = get_session_state(user_id, company_id)
    state.phone = phone.strip()
    state.phone_collected = True
    return save_session_state(state)

// session_state_manager.py session_state_to_prompt (lines 194-195)
if state.phone_collected:
    lines.append("- RULE: Phone already collected. Do NOT ask for phone or WhatsApp again.")

// response_validator.py (lines 128-130)
if phone_collected and _contains_pattern(sanitized, PHONE_ASK_PATTERNS):
    issues.append("repeated_phone_request")
    sanitized = _strip_phone_asks(sanitized)

// ai_employee_engine.py _build_user_prompt (lines 145-148)
if not phone_collected and not suppress_phone_ask:
    sales_rules += "- Ask for phone or WhatsApp if not available and purchase intent is strong.\n"
elif phone_collected:
    sales_rules += "- Phone is already collected. Do NOT ask for phone or WhatsApp again.\n"
```

7b. duplicate reply prevention

```
// main.py (lines 38-39)
PROCESSED_UPDATE_IDS: set[int] = set()
PROCESSED_UPDATE_ORDER: deque[int] = deque(maxlen=500)

// main.py _already_processed_update (lines 119-137)
def _already_processed_update(update_id: int | None) -> bool:
    if update_id is None:
        return False
    uid = int(update_id)
    if uid in PROCESSED_UPDATE_IDS:
        return True
    if len(PROCESSED_UPDATE_ORDER) >= PROCESSED_UPDATE_ORDER.maxlen:
        oldest = PROCESSED_UPDATE_ORDER.popleft()
        PROCESSED_UPDATE_IDS.discard(oldest)
    PROCESSED_UPDATE_ORDER.append(uid)
    PROCESSED_UPDATE_IDS.add(uid)
    return False

async def handle_update(...):
    update_id = update.get("update_id")
    if _already_processed_update(update_id):
        logger.info("TELEGRAM_UPDATE_DEDUPED update_id=%s", update_id)
        return

// response_validator.py (lines 118-121)
if last_reply:
    overlap = _token_overlap_ratio(sanitized, last_reply)
    if overlap >= overlap_threshold:
        issues.append(f"duplicate_response overlap={overlap:.2f}")

// ai_employee_engine.py _finalize_reply fallback (lines 247-254)
if not validation.is_valid:
    final_reply = build_fallback_reply(
        turn_route.turn_type,
        language=session.language,
        company_name=company_name,
    )
```

7c. overlap detection (>70%)

```
// response_validator.py _token_overlap_ratio (lines 59-66)
def _token_overlap_ratio(a: str, b: str) -> float:
    tokens_a = set(_normalize_for_compare(a).split())
    tokens_b = set(_normalize_for_compare(b).split())
    if not tokens_a or not tokens_b:
        return 0.0
    intersection = tokens_a & tokens_b
    return len(intersection) / max(len(tokens_a), len(tokens_b))

// response_validator.py validate_response (lines 103-121)
def validate_response(..., overlap_threshold: float = 0.70) -> ValidationResult:
    ...
    if last_reply:
        overlap = _token_overlap_ratio(sanitized, last_reply)
        if overlap >= overlap_threshold:
            issues.append(f"duplicate_response overlap={overlap:.2f}")
```

7d. language lock injection

```
// language_lock.py language_lock_prompt (lines 59-69)
def language_lock_prompt(language: str) -> str:
    if language == "nepali":
        return (
            "Language lock: The user is writing in Nepali. "
            "Reply ONLY in natural Nepali ..."
        )
    return (
        "Language lock: The user is writing in English. "
        "Reply ONLY in English ..."
    )

// ai_employee_engine.py compose_system_prompt call (lines 473-474)
    session_state_block=session_state_to_prompt(session),
    language_lock_block=language_lock_prompt(session.language),

// prompts.py sections array (lines 81-84)
sections = [
    BASE_SYSTEM_PROMPT,
    language_lock_block.strip(),
    business_block.strip(),
```

1. session_state_manager.py (full file, 201 lines)

```
from __future__ import annotations

import logging
from dataclasses import dataclass, field
from typing import Any

from memory_db import ConversationState, get_session

logger = logging.getLogger("vyapar.session_state")

@dataclass
class SessionState:
    user_id: str
    company_id: str
    language: str = "english"
    name: str | None = None
    phone: str | None = None
    location: str | None = None
    package_interest: str | None = None
    lead_stage: str | None = None
    pitch_count: int = 0
    phone_collected: bool = False
    escalation_requested: bool = False
    last_assistant_reply: str | None = None
    turn_count: int = 0
    ... [get_session_state, sync_session_state, mark_phone_collected,
        session_state_to_prompt – see repository file Z:\Vyapar AI\cursor\session_state_manager.py]
```

1. turn_router.py — route_turn signature + direct answers

```
def route_turn(text, *, session, detected_intent, sales_objection=None, company_id, language="english") -> TurnRoute:
    ...
    if _matches_any(text, AI_IDENTITY_PATTERNS):
        return TurnRoute(turn_type="company_info", suppress_catalog=True,
            direct_answer=_ai_identity_answer(company_id, language), reason="ai_identity_query")
    if _matches_any(text, COMPANY_INFO_PATTERNS) and not coverage_exclusion:
        return TurnRoute(turn_type="company_info", suppress_catalog=True,
            direct_answer=_company_info_answer(company_id, language), reason="company_name_query")
    ... [9 turn types total – see Z:\Vyapar AI\cursor\turn_router.py lines 140-237]
```

1. response_validator.py — validate_response signature

```
def validate_response(reply, *, last_reply=None, phone_collected=False,
    suppress_catalog=False, known_phone=None, overlap_threshold=0.70) -> ValidationResult:
    ... [see Z:\Vyapar AI\cursor\response_validator.py lines 103-146]
```

1. language_lock.py — detect_language + language_lock_prompt

```
def detect_language(text: str) -> str:
    ... nepali_hits / english_hits token scoring ...
    return "nepali" | "english"

def language_lock_prompt(language: str) -> str:
    ... [see Z:\Vyapar AI\cursor\language_lock.py lines 23-69]
```

1. memory_validator.py — is_high_confidence_name + validate_memory_facts

```
def is_high_confidence_name(text: str, candidate: str | None) -> bool:
    ... rejects multi "I am", reject words (indian, nepali, ...), >4 words ...

def validate_memory_facts(text: str, facts: dict[str, str]) -> dict[str, str]:
    ... name → is_high_confidence_name; phone → is_valid_phone ...
    ... [see Z:\Vyapar AI\cursor\memory_validator.py lines 43-110]
```